B.Tech. Computer Science Engineering and Artificial Intelligence and Machine Learning

COURSE PLAN: THEORY COURSE

School Name	**School of Computer Engineering**				
Course Name	**Parallel Computer Architecture and Programming**				
Course Code	**CSE 3129**	**Semester**	**V**	**Core / PE / OE**	**Core**
Name of the faculty	**Dr. Neelima Bayyapu**				

No of contact hours/week:	**L**	**T**	**P**	**C**
	3	**0**	**3**	**4**

Mapping of Course Outcomes (COs)/Course Learning Outcomes (CLOs) #

CO/ CLO	Statements	No. of Contact Hours	Marks	Program Outcomes (POs)	Program Specific Outcomes (PSOs)	Learning Outcomes #	BL* *(Recommended)*
CO1	Demonstrate a fundamental understanding of architecture-level support for parallelization and programming paradigms	5	14	1,2,11	1,2	C1	2 to 6
CO2	Design and implement shared memory programs using OpenMP	6	17	1,2,3,5,11	1,2	C1	3 to 6
CO3	Design and implement distributed memory programs using MPI	6	17	1,2,3,5,11	1,2	C1, C2	3 to 6
CO4	Design and construct CUDA programs using GPU architecture	10	27	1,2,3,4,5,11	1,2	C1, C2	3 to 6
CO5	Design optimized parallel solutions using thread and memory organization in CUDA	9	20	1,2,3,4,5,11	1,2	C1, C2	3 to 6
CO6	Applying Parallel Programming Concepts to Applications	NA	5	1,2,3,5,9	1,2	C2, C14	3 to 6
	Total	**36**	**100**				

Applicable to IET Accredited Programs Only

**Preferred BL levels are above 3 (apply)*

Course Articulation Matrix

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3	PSO4
CO1	3	1									2	3	1		3

CO2	3	2	3		1						2	3	3		3
CO3	3	2	3		1						2	3	3		3
CO4	3	2	3	2	1						2	3	3		3
CO5	3	2	3	2	1						2	3	3		3
CO6	2	2	1		1				1			3	1		2
Average Articulation Level	2.83	1.83	2.60	2.00	1.00	NA	NA	NA	1.00	NA	2.00	3	2.333	NA	2.83

Module Matrix#

	LO1	LO2	LO3	LO4	LO5	LO6	LO7	LO8	LO9	LO10	LO11	LO12	LO13	LO14	LO15	LO16	LO17	LO18
CLO1	✓																	
CLO2	✓																	
CLO3	✓	✓																
CLO4	✓	✓																
CLO5	✓	✓																
CLO6		✓												✓				

Applicable to IET Accredited Programs Only. Fill with B or C

Delivery and Assessment Plan of LOs#

Learning Outcome (LO) mapped to the course		**Delivery and assessment Plan**
LO	**LO statement**	
CLO1	Demonstrate a fundamental understanding of architecture-level support for parallelization and programming paradigms	Get addressed during lecture delivery And Assesment is through midterm and endterm exam
CLO2	Design and implement shared memory programs using OpenMP	Get addressed during lecture delivery And Assesment is through midterm and endterm exam and lab evaluation of respective programs
CLO3	Design and implement distributed memory programs using MPI	Get addressed during lecture delivery And Assesment is through midterm and endterm exam and lab evaluation of respective programs

CLO4	Design and construct CUDA programs using GPU architecture	Get addressed during lecture delivery And Assesment is through midterm and endterm exam and lab evaluation of respective programs
CLO5	Design optimized parallel solutions using thread and memory organization in CUDA	Get addressed during lecture delivery And Assesment is through midterm and endterm exam and lab evaluation of respective programs
CLO6	Applying Parallel Programming Concepts to Applications	Get addressed during lecture delivery And Mini project evaluation

Applicable to IET Accredited Programs Only

ICT Tools used in delivery and assessment

Sl. No	Name of the ICT tool used	Details of how it is used
1	Power point presentation	PPT used to teach
2	Microsoft Teams	Assignments made to upload in teams
3	LMS	Lecture Notes sharing and evaluations will be conducted

Assessment Plan (As communicated from o/o AD-A, in every odd semester)

IN – SEMESTER ASSESSMENTS									
Sl. No.	**Assessment Mode**		**Assessment Method**	****Time Duration**	****Marks**	**** Weightage**	**Typology of Questions** *(Recommended)*	****Schedule**	****Topics Covered**
1	**MISAC (Through Integrated Lab Assessment)**	**1**	**Continuous Lab evaluations**	**Lab1 to Lab 10**	**8M (40Marks from Lab converted to 8M)**	**4 Marks per lab for 10 Labs = 40Marks**	L3, L4	Respective Labs	OpenMP, MPI and CUDA
		2	**Lab Exam**	**After Lab 8**	**4M (20Marks from Lab converted to 4M)**	Lab exam based on all COs= 20M	L3, L4	Lab 8 or Lab 9	OpenMP, MPI and CUDA
		3	**Mini Project**	**Lab 3 to Lab 10..Evaluation in Lab 5, 10, and 12**	**8M (40Marks from Lab converted to 8M)**	Mini-project evaluation = 40M (Detailed evaluation strategy will be shared)	L3, L4	Lab 5, Lab 10 and Lab 12	OpenMP, MPI and CUDA
2	**MISAC**	**1**	**Mid term Theory Exam**	**1.5 hours**	**30 Marks**	25 M theory and 5M based on LAb	L4, L5	25th Sep. to 1st Oct, 2026	OpenMP, MPI and theory and lab
END – SEMESTER ASSESSMENT									
1	**Regular/Make–Up Exam**			180 Mins	50	Answer all 5 full questions of 10 marks each. Each question can have 3 parts of 2/3/4/5/6 marks.	Bloom's taxonomy (BT) level of the question should be L3 and above.	17th week of the semester	Comprehensive examination covering full syllabus.

*** Individual faculty will be entering the details*

**** Individual faculty shall identify the assessment method from FISAC Assessment method (Table 1 below) and fill in the details.*

***NOTE:** Information provided in the Table 1 is as per the In-semester assessment plan notified by Associate Director (Academics).*

Lesson Plan

L No	Topics	CO Addressed
0	*To address OBE philosophy towards NBA and IET accreditation followed by a discussion on Course Plan, Course Outcomes, CO-PO Mapping and Blooms Taxonomy, with prerequisites for the course, if any.*	
1	**Introduction to Parallel Computing :** Need of Parallelism; Forms of Parallelism, Flynn's taxonomy: SISD, SIMD, MISD, MIMD	CO1
2	Moore's Law and Multicore; Amdahl's Law	CO1
3	Speedup and performance; Programming paradigms: Comparison between Shared and Distributed Memory Programming Paradigms	CO1
4	Introduction to Parallel Random Access Machine (PRAM) Architecture Model: Characteristics of PRAM Models; Types of PRAM Models	CO1
5	Introduction to Parallel Random Access Machine (PRAM) Architecture Model: Advantages and Limitations of PRAM Models	CO1
6	**Shared Memory Programming Using OpenMP:** Introduction to OpenMP: What is OpenMP?, Shared-memory programming: Advantages and limitations, OpenMP programming model	CO2
7	Fork-join execution and thread-based parallelism, Compiling OpenMP Programs: Enabling OpenMP and Running programs, OpenMP Compiler Directives: parallel, for, sections, single, master	CO2
8	Data Sharing: Shared variables; Private variables; firstprivate; lastprivate; reduction	CO2
9	Synchronization: Barrier; Critical sections; Atomic operations. Runtime Functions: omp_get_thread_num(); omp_get_num_threads(); omp_set_num_threads(); omp_get_wtime(). Environment Variables: OMP_NUM_THREADS; OMP_SCHEDULE; OMP_DYNAMIC	CO2
10	Work Sharing: for directive; sections; single; Combined Parallel and Work-sharing constructs.	CO2
11	Scheduling (static, dynamic, guided)	CO2
12	**Distributed Memory Programming Using MPI:** Message-Passing Programming, MPI Basics and Introduction to Distributed-Memory Architectures	CO3
13	Foundational Concepts of the MPI Standard; Compiling and Running MPI Programs	CO3
14	Point-to-Point Communication: Send/Recv Between Two Processes; Synchronous and Buffered Send	CO3
15	Collective Communication: Broadcast, Scatter, Gather and Reduce Operations	CO3
16	Collective Data Movement and Reduction; Introduction to Allgather and Alltoall	CO3
17	Measuring the Execution Time of MPI Programs; MPI Error-Handling Functions	CO3
18	**Accelerated Programming Using GPU:** GPU Architecture—Transition from CPU to GPU	CO4
19	Architecture of a Modern GPU	CO4
20	CUDA Program Structure and Vector–Vector Addition	CO4
21	Device Global Memory and Data Transfer	CO4
22	CUDA Kernel Functions and Threads	CO4
23	CUDA Thread Organization	CO4
24	Assigning Resources to Blocks and Querying Device Properties	CO4
25	Sequential and Parallel 1D Convolution	CO4
26	Sequential Sparse Matrix–Vector Multiplication and Parallel SpMV Using CSR	CO4
27	CUDA Atomic and Arithmetic Functions	CO4
28	**High-Performance GPU Programming Using Optimizations:** Introduction to GPU Performance Optimization	CO5

29	Mapping Threads to Multidimensional Data	CO5
30	Matrix–Matrix Multiplication: A More Complex CUDA Kernel	CO5
31	Synchronization and Transparent Scalability	CO5
32	CUDA Device Memory Types; Constant Memory and Caching	CO5
33	Tiled 1D Convolution with Halo Elements	CO5
34	Importance of Memory-Access Efficiency	CO5
35	Reducing Global-Memory Traffic and Tiled Matrix–Matrix Multiplication	CO5
36	Thread Scheduling and Latency Tolerance	CO5

Self Directed Learning (SDL)

Sl. No.	TOPIC	No. of learning hours	CO
1	Introduction to Parallel Computing – Study the need for parallel computing, Flynn's Taxonomy and multicore evolution. Prepare a comparison chart of SISD, SIMD, MISD and MIMD architectures. NPTEL: Parallel Computer Architecture – https://nptel.ac.in/courses/106106234	3	CO1
2	Performance Analysis – Solve problems on Amdahl's Law, Gustafson's Law, speedup, efficiency and scalability. Compare serial and parallel execution for different processor counts. NPTEL: https://nptel.ac.in/courses/106106234	3	CO1
3	Programming Paradigms – Compare shared-memory, distributed-memory and GPU programming paradigms. Prepare a report identifying suitable application domains for OpenMP, MPI and CUDA. Reference: OpenMP https://www.openmp.org/ ; OpenMPI https://www.open-mpi.org/ ; NVIDIA CUDA https://developer.nvidia.com/cuda-zone	3	CO1
4	Introduction to OpenMP – Install OpenMP, compile sample programs and experiment with parallel, for, sections and single directives. NPTEL: Programming in Modern C++ (OpenMP Modules) https://nptel.ac.in/courses/106102064 ; OpenMP: https://www.openmp.org/specifications/	3	CO2
5	Data Sharing and Synchronization – Implement shared/private variables, reduction, barrier, critical and atomic directives. Compare correctness and execution time. OpenMP Examples: https://www.openmp.org/resources/openmp-examples/	3	CO2
6	Scheduling and Runtime Functions – Compare static, dynamic and guided scheduling using matrix multiplication or numerical computation. Measure performance using omp_get_wtime(). OpenMP API: https://www.openmp.org/specifications/	3	CO2
7	MPI Basics – Install OpenMPI and execute "Hello World" and basic communication programs using multiple processes. NPTEL: Parallel Computer Architecture https://nptel.ac.in/courses/106106234 ; OpenMPI: https://www.open-mpi.org/doc/	3	CO3
8	Point-to-Point Communication – Implement MPI_Send() and MPI_Recv() for vector addition or matrix operations. Observe process communication. MPI Tutorial: https://mpitutorial.com/	3	CO3
9	Collective Communication – Implement Broadcast, Scatter, Gather and Reduce operations. Compare communication overhead with point-to-point communication. MPI Standard: https://www.mpi-forum.org/docs/	3	CO3
10	GPU Architecture – Study GPU architecture and compare CPU vs GPU execution models. Draw the organization of Streaming Multiprocessors and CUDA cores. NPTEL: Parallel Computer Architecture	3	CO4

	https://nptel.ac.in/courses/106106234 ; CUDA: https://developer.nvidia.com/cuda-zone		
11	CUDA Programming Basics – Implement vector addition using CUDA kernels and analyze thread, block and grid organization. CUDA Toolkit: https://developer.nvidia.com/cuda-toolkit ; CUDA C Programming Guide: https://docs.nvidia.com/cuda/cuda-c-programming-guide/	3	CO4
12	Memory Management in CUDA – Study host-device memory allocation, memory transfer and kernel launch configuration. Implement matrix addition on GPU. CUDA Documentation: https://docs.nvidia.com/cuda/	3	CO4
13	GPU Memory Optimization – Study global, shared, local, constant and texture memory. Compare execution time before and after shared-memory optimization. CUDA Best Practices: https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/	3	CO5
14	Warp Execution and Thread Organization – Study warp scheduling, thread divergence and occupancy. Analyze CUDA kernel performance using NVIDIA Nsight Compute. NVIDIA Nsight: https://developer.nvidia.com/nsight-tools	3	CO5
15	Memory Optimization using Tiling – Implement tiled matrix multiplication or convolution using shared memory and compare speedup with the naive implementation. CUDA Samples: https://github.com/NVIDIA/cuda-samples	3	CO5
16	Parallel Programming Case Study / Mini Project – Study a research paper or develop a mini-project using OpenMP, MPI or CUDA (e.g., matrix multiplication, image processing, reduction, histogram computation). Compare serial and parallel performance and prepare a report. NPTEL: Parallel Computer Architecture https://nptel.ac.in/courses/106106234 ; IEEE Xplore: https://ieeexplore.ieee.org ; GitHub CUDA Samples: https://github.com/NVIDIA/cuda-samples	9	CO6
	Total Number of learning hours	54	

***The SDL topics may be from the Research Literature/ Case Studies NPTEL/Coursera/any MOOC-based material Virtual Lab, etc.**

****One credit is equivalent to 30 hours of learning. (4 credits course =120 hrs learning ; 48+72=120 hrs of learning)**

Faculty members teaching the course (if multiple sections exist):

Faculty Name	Section	Signature	Faculty Name	Section	Signature
Dr. Ashutosh Holla B	CSE A		Dr. Aarabhi Putty	AIML A	
Dr. Neelima Bayyapu	CSE B		Dr. Sanjana Kavatagi	AIML B	
Dr. Arul Elango	CSE C		Ms. Jayashree Hegde K	AIML C	
Dr. Sandhya Parasnath Dubey	CSE D				

References:

Textbooks	1. V.Rajaraman, C. Siva Ram Murthy, "Parallel Computers Architecture and Programming" Prentice-Hall India, 2016
	2. Barbara Chapman, Gabriele Jost, Ruud Van Der Pas, " Using OpenMP Portable Shared Memory Parallel Programming", MIT Press
	3. D. Kirk and W. Hwu , "Programming Massively Parallel Processors –A Hands-on approach", Elsevier Inc.,(2e), 2013

	4. Michael J. Quinn, "Parallel Programming in C with MPI and OpenMP", McGraw Hill Edition, 2003
	5. Michael J. Quinn, "Parallel Computing" McGraw Hill Edition, 2005
	6. Kai Hwang and Faye A. Briggs, Computer Architecture and Parallel Processing, (2e), TMH Private Ltd., 2012.
NPTEL/Coursera/any MOOC-based material	1. https://www.open-mpi.org/doc/ 2. https://www.openmp.org/ 3. https://developer.nvidia.com/ 4. nptel.ac.in/courses/112105293

Submitted by:

(Signature of the faculty)
Date:
Approved by:

(Signature of the Dean)